

Getting Started (Native Swift Project)

This replaces nurtura-ios-getting-started.pdf for this folder. That document describes an Expo/React Native/Convex-JS project (npm, expo, a different GitHub repo). This folder is a completely different, native Swift/SwiftUI implementation. The two are not interchangeable — follow this guide for what's actually in ~/iOS/Nurtura.

What this project actually is

A native iOS 17 / watchOS 10 app written in Swift 5.9 + SwiftUI, managed as a raw Swift Package (Package.swift) rather than an .xcodeproj. There is no package.json, no node_modules, no Expo, and no npm/yarn anywhere in this folder — none of the tools in the other getting-started PDF apply here.

Before you start

Xcode 15 or newer (Mac App Store → search “Xcode” — free, ~12GB). That's the only tool required; Swift 5.9 and the iOS/watchOS SDKs ship inside it.

Opening the project

```
cd ~/iOS/Nurtura
open Package.swift
```

Xcode opens it as a Swift Package (you'll see the existing .swiftpm/xcode/package.xcworkspace — that's expected, it's Xcode's own generated state for a package opened this way, not something to delete).

Current status: the full app does not build yet

Three concrete blockers, found by reading every source file in this folder:

- **RootView.swift's MainTabView switches over DashboardView(), CareRecipientsView(), ScheduleView(), MessagesView(), and SettingsView() — none of these five types exist anywhere in the codebase. The corresponding folders (Views/Dashboard, Views/CareRecipients, Views/Schedule, Views/Messages, Views/Settings) exist but are empty.**
- **NurturaApp.swift's AppDelegate calls WatchConnectivityManager.shared, HealthKitManager.shared, NotificationManager.shared, and PushNotificationManager.shared — none of these four classes exist anywhere in the codebase either.**
- **Package.swift itself was pointing at Sources/NurturaShared and Tests/NurturaSharedTests, neither of which existed — the manifest was disconnected from the real Nurtura/ and NurturaTests/ folders. This has been fixed as part of this session (see Change Log below).**

What works right now

The pure logic layer — Models.swift, AuthManager.swift, ConvexAPI.swift — is now wired into a real, buildable library target (NurturaShared) with a passing-by-design unit test suite. You can run just that slice today:

```
cd ~/iOS/Nurtura
swift test
```

The first time this touches the Keychain (KeychainManager tests) macOS may show a one-time access prompt for an unsigned test binary — that's normal, allow it.

Backend

AppConstants.convexURL in Models.swift points to <https://optimistic-tapir-431.convex.cloud>. The other (Expo) getting-started doc lists two different deployments — [capable-stork-178 \(prod\)](#) and [successful-shrimp-557 \(dev\)](#) — neither of which matches. Confirm which Convex deployment this native app is actually supposed to talk to before wiring up real network calls; right now it may be pointed at a stale or placeholder URL.

Path to a running app

- Write the five missing view files (Dashboard, CareRecipients, Schedule, Messages, Settings).
- Write or stub the four missing manager classes (WatchConnectivityManager, HealthKitManager, NotificationManager, PushNotificationManager) — stubs are enough to get a build passing before real HealthKit/WatchConnectivity integration is written.
- Confirm the correct Convex URL and update AppConstants if needed.
- Once it builds, run it via Xcode's Run button with an iPhone Simulator destination selected.